

Eficiência de agentes de programação: qualidade, custo e latência

Isaac

Resumo—Este artigo delimita um protocolo para desenvolver e avaliar uma abordagem de execução de agentes de programação que melhore a relação entre custo e qualidade em comparação com um modelo em seu harness padrão, sob restrição de latência. A avaliação considera tanto economia com preservação da taxa de resolução quanto reduções de custo que justifiquem perdas delimitadas nessa taxa. Evidências sobre processos, contexto, verificadores e validade de benchmarks fundamentam comparações pareadas com agentes de modelo único, um modelo forte com técnicas de economia de tokens e uma política simples de escalonamento. A investigação distingue composição fixa e roteamento de modelos, seleção e distribuição de contexto e ativação de etapas do processo, avaliando suas interações. Taxa de resolução, custo monetário e tempo serão medidos separadamente, incluindo execuções malsucedidas e intervenções auxiliares. Como ainda não há resultados experimentais, os mecanismos, incluindo revisão e paralelização, permanecem hipóteses; o protocolo estabelece controles para atribuir seus efeitos sem antecipar uma arquitetura superior.

Palavras-chave—agentes de programação, engenharia de contexto, roteamento de modelos, scaffolding, avaliação de benchmarks

I. INTRODUÇÃO

Em agentes de programação, a taxa de resolução informa se uma tarefa foi concluída segundo o verificador, mas, sozinha, não informa quanto foi necessário gastar para obtê-la. Repetições, chamadas adicionais e revisão podem alterar simultaneamente a probabilidade de sucesso e o consumo de recursos. Kapoor et al. identificam a ênfase exclusiva em acurácia como uma limitação da avaliação de agentes e formulam a otimização conjunta de acurácia e custo; seus experimentos mostram que considerar custo pode mudar a interpretação de estratégias que parecem superiores quando se observa apenas a taxa de sucesso [1]. Para decisões de engenharia, portanto, a comparação precisa mostrar a qualidade alcançada junto com os recursos consumidos, em vez de tratar qualquer ganho de resolução como vantagem suficiente.

A necessidade de avaliar qualidade e recursos conjuntamente torna o harness parte da condição experimental. Em um estudo controlado com 50 tarefas do Terminal-Bench Pro e os modelos Qwen 3.6 Plus e MiniMax M2.5, as diferenças pareadas de taxa de resolução entre harnesses variaram de 0 a 8 pontos percentuais. Os intervalos de confiança incluíram zero na maioria das comparações, o que não demonstra equivalência entre os harnesses. Mantido fixo cada modelo, o OpenCode consumiu cerca de 40 vezes mais tokens por tarefa resolvida do que o Goose. Esse contraste reforça que a taxa de resolução, isoladamente, omite diferenças substanciais de consumo mesmo em um contexto experimental controlado

[2]. Como a avaliação abrangeu apenas dois modelos e três harnesses, o fator de 40 vezes demonstra a possibilidade de grande variação, mas não estima a magnitude do efeito em outros modelos, sobretudo modelos mais recentes ou mais capazes. Tokens tampouco equivalem a um custo monetário fixo, pois tarifas, cache e categorias de cobrança variam.

Além de afetar custo e desempenho, a infraestrutura pode determinar se uma alteração produzida pelo agente chega sequer a ser testada. O Claw-SWE-Bench evidencia esse problema na interface entre agente e avaliador. Em sua configuração mínima, o modelo precisava escrever um *diff* unificado na resposta final; erros em caminhos, cabeçalhos ou linhas de contexto podiam impedir a aplicação do patch antes da execução dos testes. Na configuração completa, o agente editava os arquivos do repositório, e a camada de integração calculava o *diff* em relação ao estado inicial, removia artefatos alheios à solução e produzia o patch no formato aceito pelo avaliador. Com o mesmo modelo GLM 5.1 e as mesmas 350 tarefas, a taxa de resolução passou de 19,1% para 73,4%, enquanto a proporção de patches não aplicáveis caiu de 69,1% para menos de 1,5% [3]. O contraste não atribui esse ganho apenas à extração do patch, pois a configuração completa também modificou outros elementos do adaptador. Ele demonstra, contudo, que a pontuação pode confundir a capacidade de resolver a tarefa com a capacidade de satisfazer o contrato de entrega. A unidade de comparação deve, assim, identificar modelo, harness, ferramentas, orçamento e procedimento de entrega.

O efeito de um scaffold também pode variar com o modelo e a tarefa. Em um estudo controlado com GAIA, os efeitos diferiram entre famílias de modelos e níveis de dificuldade; a expectativa de que o scaffold se tornaria sistematicamente menos relevante com modelos mais capazes não se confirmou no nível mais difícil avaliado [4]. Esse resultado justifica medir a interação entre modelo e scaffold, em vez de pressupor que a mesma organização de agentes terá valor uniforme.

Controlar o harness ainda não basta para assegurar que a pontuação represente qualidade de engenharia de software. Gorinova et al. argumentam que benchmarks de programação podem confundir o efeito do modelo com o do harness, vincular a correção à solução de referência e fornecer pouco sinal sobre a origem das falhas [5]. Por se tratar de um artigo de posição, essas críticas não quantificam a frequência dos problemas nos conjuntos considerados aqui. Elas exigem, contudo, explicitar o que o verificador mede e registrar falhas de entrega e de execução separadamente, antes de extrapolar uma taxa de resolução para qualidade do software produzido.

Mesmo uma comparação controlada pode perder validade se o benchmark favorecer o reconhecimento de tarefas já conhecidas. Prathifkumar et al. avaliaram uma etapa anterior à correção: a identificação dos arquivos modificados na solução de cada issue. Com apenas o texto da issue, sem acesso ao código, à lista de nomes dos arquivos ou à estrutura do repositório, Claude 3.5 e Claude 3.7 identificaram todos esses arquivos em, respectivamente, 65% e 63,2% das 500 tarefas do SWE-bench Verified. Para comparação, o estudo aplicou o mesmo teste a 500 issues de cinco repositórios Python do BeetleBox, ausentes do Verified, e a dois conjuntos mais recentes do SWE-rebench, coletados em janeiro e setembro de 2025. No BeetleBox, as taxas foram 12,2% e 12%; nos dois conjuntos do SWE-rebench, variaram de 8% a 19,27%. Trata-se de localização de arquivos, não de resolução das issues. A capacidade de localizar todos os arquivos sem consultar o repositório, somada à queda acentuada nos outros conjuntos, constitui um indício de exposição prévia às tarefas do Verified [6]. Como os conjuntos também diferem em repositórios e tarefas, essa comparação não identifica o conteúdo do treinamento nem comprova memorização de patches específicos. Ela justifica registrar a procedência e a data das tarefas, separar desenvolvimento e avaliação e interpretar com cautela pontuações em benchmarks públicos antigos.

A seleção dos benchmarks deve, portanto, combinar correspondência com a tarefa-alvo, procedência das instâncias, cobertura de contextos e verificadores reproduzíveis. O SWE-bench-Live será o conjunto principal para resolução de issues em repositórios; o Multi-SWE-bench examinará a variação entre linguagens; e o Terminal-Bench fornecerá evidência complementar sobre execução em linha de comando [7]–[9]. Esses conjuntos medem tarefas diferentes e suas pontuações serão interpretadas separadamente. A maior atualidade de um conjunto reduz uma possível fonte de exposição prévia, mas não demonstra ausência de contaminação.

Após escolher os conjuntos, a escolha de um subconjunto depende do objetivo: comparar a ordenação de configurações ou estimar a taxa de resolução no benchmark completo. Ndzomga seleciona tarefas cuja taxa histórica de sucesso está entre 30% e 70%, pois tarefas resolvidas por quase todos ou por quase ninguém oferecem pouca informação para distinguir agentes. Nos cenários avaliados, esse filtro preserva bem a ordenação, medida pelas correlações de Spearman e Kendall, mesmo quando a previsão das pontuações no conjunto completo, medida por R^2 , perde precisão sob mudanças de scaffold [10]. O subconjunto pode, portanto, orientar a triagem de configurações, mas sua taxa de resolução não estima diretamente a taxa no benchmark inteiro. A avaliação confirmatória das configurações selecionadas usará tarefas reservadas, não empregadas em seu ajuste ou seleção.

A literatura de arquiteturas oferece mecanismos para investigar essa relação econômica. Agentless mostra que um processo fixo pode resolver issues sem delegar ao modelo a escolha livre das etapas; Jiang et al. comparam resolução com contexto amplo e separação entre seleção de arquivos e reparo; e icat-agent condiciona a exploração preliminar às

informações disponíveis na descrição da issue [11]–[13]. Esses resultados não estabelecem uma ordem universal entre processos simples, composições de modelos e sistemas multiagentes. Eles motivam avaliar conjuntamente qual modelo executa cada atividade, qual contexto recebe e quando uma etapa adicional compensa seus recursos.

O objetivo desta pesquisa é desenvolver e avaliar uma abordagem que ofereça uma relação mais favorável entre custo e qualidade do que a execução de um modelo em seu harness padrão, mantendo o aumento de latência dentro de um limite aceitável. A pergunta central é se essa abordagem reduz o custo preservando a taxa de resolução ou oferecendo uma economia que justifique uma perda delimitada nessa taxa. O protocolo compara o sistema completo com agentes de modelo único, inclusive um modelo forte com técnicas de economia de tokens, e com uma política simples de escalonamento. A execução por modelos de menor custo com ajuda seletiva de modelos mais capazes permanece uma hipótese de solução. Também serão investigadas a composição inversa, com preparação de contexto por um modelo e implementação por outro, e a ativação seletiva de etapas. A seção seguinte estabelece comparações e critérios de avaliação; a investigação dos mecanismos distingue evidência anterior de hipóteses ainda abertas, sem pressupor que roteamento ou múltiplos agentes sejam necessários para obter o benefício procurado.

II. PROTOCOLO DE AVALIAÇÃO

A. Objetivo e desenho comparativo

A avaliação responderá se a abordagem desenvolvida oferece uma relação entre custo e qualidade mais favorável do que a execução de um modelo em seu harness padrão, sob uma restrição de latência. A referência prática será uma configuração documentada do modelo forte no harness selecionado, como Claude Code ou Codex, preservando os recursos nativos de execução. Esses nomes identificam candidatos a referência; o protocolo não pressupõe equivalência entre eles nem exige usar um modelo em um produto que não o suporte. Cada conclusão será vinculada à combinação efetivamente avaliada de modelo, harness e parâmetros.

O contraste principal será entre sistemas completos. Cada condição receberá as mesmas tarefas e estados iniciais, com permissões, acesso a ferramentas externas, testes disponíveis e tetos de recursos comparáveis. A organização interna do trabalho poderá variar, pois constitui parte da intervenção. Comparações que isolem um componente manterão os demais fatores fixos; quando isso não for possível, a diferença será atribuída ao conjunto da configuração. A ordem das execuções será aleatorizada em blocos por tarefa para reduzir a associação entre condição e horário de acesso ao serviço.

B. Condições e contrastes

Serão avaliadas cinco condições:

- 1) **Modelo forte no harness padrão:** referência principal de resolução, custo e latência, com versão e configuração de execução registradas.

- 2) **Modelo barato no harness de referência compatível:** controle para medir o benefício e a perda de resolução obtidos pela substituição do modelo. O harness será mantido quando houver suporte; diferenças necessárias de harness serão documentadas e impedirão atribuir o contraste exclusivamente ao modelo.
- 3) **Modelo forte com economia de tokens:** o mesmo modelo e harness da primeira condição, acrescidos de técnicas explicitamente selecionadas para reduzir consumo. As técnicas serão definidas na investigação dos mecanismos e congeladas antes da avaliação final. Seu efeito será medido em custo monetário, resolução e tempo; reduzir tokens, isoladamente, não caracterizará benefício.
- 4) **Modelo barato com escalonamento simples:** execução inicial pelo modelo barato e intervenção do forte segundo uma regra fixa, baseada apenas em sinais disponíveis durante a execução. A regra, os limites e a transferência de contexto serão documentados.
- 5) **Abordagem desenvolvida:** configuração selecionada na etapa de desenvolvimento, que poderá empregar um ou mais modelos e mecanismos de economia, verificação ou coordenação.

A quinta condição será comparada à primeira para responder à pergunta principal. Os contrastes com a segunda, a terceira e a quarta avaliarão se o benefício também pode ser obtido por alternativas mais simples. Superar apenas a referência padrão não demonstrará superioridade sobre esses controles. O contraste entre a primeira e a terceira condições medirá especificamente a contribuição das técnicas de economia aplicadas ao modelo forte.

As otimizações nativas da referência serão preservadas; a terceira condição acrescentará intervenções identificáveis sobre essa base. Todas as condições terão acesso à mesma informação externa permitida, sem testes reservados ou soluções de referência. Configurações ajustáveis receberão oportunidades documentadas de ajuste nas tarefas de desenvolvimento. Ablações corresponderão aos componentes efetivamente adotados e às alegações sobre seus efeitos. Um controle adicional de repetição simples será incluído se repetição for um mecanismo central, distinguindo o efeito da política do benefício de gastar mais inferência [1].

C. Contrastes de contexto, composição e processo

O desenvolvimento examinará conjuntamente modelo e estratégia de contexto em um conjunto limitado de variantes. Para a composição por atividade, o piloto cruzará, quando tecnicamente compatíveis, dois modelos nas etapas de seleção de contexto e resolução. As combinações com o mesmo modelo nas duas etapas serão controles para as combinações heterogêneas, mantendo representações, ferramentas, amostragem de candidatos e verificação comparáveis. Uma configuração de execução direta permitirá verificar se o benefício decorre da separação em etapas. Para isolar o resolvidor, serão reutilizados os mesmos contextos selecionados; para isolar o seletor, será mantido o resolvidor. O custo da seleção integrará

a configuração completa, mesmo quando seus artefatos forem reutilizados nos contrastes diagnósticos.

Uma composição fixa não será denominada roteamento dinâmico. Se houver escolha condicional de modelos, a política será comparada com atribuições fixas e com o escalonamento simples. Analogamente, uma política de ativação de exploração dedicada será comparada com as alternativas de sempre executar e sempre dispensar essa etapa, mantendo as possibilidades de inspeção durante o reparo. Gatilhos dependerão apenas de informações acessíveis ao agente, como descrição da issue, evidências recuperadas e falhas observadas. Soluções de referência, arquivos do patch oficial e resultados dos testes reservados não poderão orientar decisões operacionais.

Intervenções de contexto serão descritas por sua fonte, granularidade, extensão, ordenação, atualização e regras de compartilhamento. Se forem adotadas skills, serão registrados conteúdo, versão, mecanismo de carregamento e critério de seleção; os controles distinguirão ausência de skill, disponibilização fixa e seleção condicional, conforme a alegação investigada. Para isolamento de validação, o contraste especificará quais informações são fornecidas ao revisor e ao editor, mantendo evidências executáveis e orçamento comparáveis. Tais contrastes serão exigidos apenas para mecanismos efetivamente adotados, evitando um produto cartesiano de todas as variantes. O piloto selecionará as comparações confirmatórias antes da avaliação reservada.

D. Benchmarks

O SWE-bench consolidou a avaliação de agentes que recebem uma issue e um repositório e devem produzir um patch verificável por execução de testes [14]. O SWE-bench-Live será o conjunto principal para esse tipo de tarefa: sua coleção atualizável reúne issues mais recentes, de um número maior de repositórios, com ambientes executáveis [7]. A atualidade reduz uma fonte potencial de exposição prévia das tarefas, mas não comprova ausência de contaminação. Tampouco uma diferença de pontuação entre conjuntos demonstra, isoladamente, memorização: a dificuldade e a composição das tarefas também podem variar.

O Multi-SWE-bench complementarà a avaliação por incluir 1.632 tarefas verificadas em Java, TypeScript, JavaScript, Go, Rust, C e C++ [8]. O estudo desse conjunto relata diferenças de desempenho entre linguagens e entre métodos de agente. O SWE-PolyBench também documenta desempenho desigual e distribuição não uniforme de tarefas por linguagem, reforçando a necessidade de apresentar resultados desagregados [15]. Python aparece nas comparações publicadas do Multi-SWE-bench por meio de tarefas de outro conjunto; não será contado como uma das sete linguagens que compõem suas 1.632 instâncias.

O Terminal-Bench será a terceira avaliação, voltada à execução de tarefas por agentes em ambientes de linha de comando. Sua publicação acadêmica apresenta o Terminal-Bench 2.0 e um protocolo de verificação por tarefa [9]. O projeto mantém versões posteriores; a versão 4.0 alterou tarefas, verificadores e recursos de execução [16]. A versão e o identificador do

conjunto serão fixados antes dos experimentos. Como suas tarefas abrangem também domínios além da engenharia de software, seus resultados serão interpretados como evidência complementar sobre atuação em terminal, e não como taxa de resolução de issues.

A inclusão de cada instância exigirá ambiente reproduzível, instruções suficientes e um verificador aplicável. Auditorias do SWE-bench Verified e do SWE-bench Pro encontraram, respectivamente, problemas de exposição de soluções e de adequação dos testes, e problemas de qualidade em tarefas do conjunto Pro [17], [18]. Esses achados motivam a auditoria e o registro de problemas nas instâncias usadas; não serão extrapolados automaticamente para o SWE-bench-Live, o Multi-SWE-bench ou o Terminal-Bench. Não haverá agregação das pontuações dos três benchmarks em um escore único.

E. Execução, entrega e registro de falhas

Cada execução começará em ambiente isolado no estado especificado pela tarefa. Nos benchmarks de issues, o agente editará os arquivos do repositório; após o encerramento, um coletor comum produzirá o patch em relação ao commit inicial, removendo apenas artefatos previamente especificados. Serão preservados a cópia de trabalho final e o patch submetido. O coletor, suas regras e a versão do avaliador serão comuns às condições, evitando introduzir exigências diferentes de serialização da solução [3]. No Terminal-Bench será aplicado o contrato de avaliação da versão selecionada.

O encerramento ocorrerá por entrega, abandono ou esgotamento de um limite de tempo, gasto ou passos. Tentativas internas e intervenções auxiliares integrarão a mesma execução e consumirão seu orçamento. Repetições experimentais começarão novamente no estado inicial e serão registradas separadamente; o melhor resultado entre repetições não será usado como resultado de uma execução comum. O avaliador reservado será acionado após o encerramento e não fornecerá sinais para novas tentativas do agente.

Nas condições que empregarem verificação interna, serão especificados o verificador, as evidências acessíveis e a regra que transforma seu resultado em aceitação, reparo, nova tentativa, escalonamento ou encerramento. Os registros distinguirão intervenções que apenas julgam uma solução daquelas que orientam ou executam alterações. Limiares e regras serão definidos no desenvolvimento e congelados antes da avaliação final. Chamadas de revisão e testes internos integrarão custo e latência, permitindo avaliar a política completa, inclusive quando detectar uma falha não resultar em reparo bem-sucedido.

Ausência de entrega, patch inválido, falha nos testes e esgotamento de recursos contarão como insucesso quando decorrerem da configuração avaliada. A causa e a etapa serão registradas: localização, edição, entrega, teste, verificação, escalonamento ou parada. Falhas comprovadamente externas, como indisponibilidade de serviço ou ambiente corrompido, terão tratamento separado, segundo critérios e limite de reexecução fixados no piloto e aplicados igualmente às condições. Os registros originais e os gastos dessas ocorrências serão

preservados, inclusive quando não entrarem na estimativa principal. Nenhuma falha será descartada apenas por produzir resultado desfavorável.

Serão registrados versões de modelos, harnesses e adaptadores, parâmetros de geração, prompts, permissões, limites, datas, chamadas, uso de tokens, ações de ferramenta, testes internos, decisões de escalonamento e motivos de encerramento. Procedência e datas das tarefas serão preservadas. A separação entre desenvolvimento e avaliação final impedirá que os resultados reservados sejam usados para selecionar configurações.

Quando houver recuperação estrutural ou índices, sua construção usará exclusivamente o estado inicial permitido do repositório. Atualizações acompanharão as alterações realizadas durante a execução, sem acesso a commits de solução ou testes reservados. Serão registrados versão do índice, consultas, trechos retornados e conteúdo efetivamente fornecido a cada etapa. Para distinguir aquecimento de economia recorrente, custos e tempos de preparação serão apresentados separadamente e incorporados ao cenário de primeira utilização. Qualquer amortização entre tarefas explicitará o número de usos, a validade do índice e os custos de atualização; o compartilhamento de cache ou índice entre condições não poderá favorecer apenas uma delas.

F. Métricas

Para um benchmark b , seja I_b o conjunto de tarefas, com tamanho n_b , e K o número comum de repetições experimentais por tarefa e condição. Em cada execução r , $s_{i,c,r} = 1$ indicará aprovação pelo verificador oficial e $s_{i,c,r} = 0$ indicará insucesso. A taxa de resolução será

$$\hat{R}_{b,c} = \frac{1}{n_b K} \sum_{i \in I_b} \sum_{r=1}^K s_{i,c,r}. \quad (1)$$

Essa taxa mede correção segundo o protocolo do benchmark. Nos conjuntos de issues, serão exigidos os testes de correção e de ausência de regressão previstos pelo respectivo avaliador [14]. A taxa de aplicação de patches e a fração de testes aprovados serão diagnósticos, sem converter aprovação parcial em resolução completa [19].

O custo de API de uma execução incluirá planejamento, execução, revisão, resumos, escalonamentos e tentativas internas, inclusive chamadas malsucedidas cobradas. Para cada chamada j , sejam $u_{j,k}$ os tokens da categoria de cobrança k , $p_{m_j,k}$ o preço por milhão de tokens do modelo empregado e f_j eventuais tarifas fixas:

$$C_{i,c,r}^{\text{API}} = \sum_{j \in J_{i,c,r}} \left(\sum_k \frac{u_{j,k} p_{m_j,k}}{10^6} + f_j \right). \quad (2)$$

As categorias de entrada, cache e saída serão mutuamente exclusivas. Moeda, data, fonte das tarifas e chamadas sem informação de cobrança serão registradas; dados ausentes não serão tratados como custo zero. O custo calculado será confrontado com a cobrança quando disponível. Para referências acessadas por assinatura, eventual valoração por tarifas de API

será identificada como estimativa, separada do desembolso da assinatura, e só será utilizada se houver dados suficientes de uso e preços verificáveis.

A medida monetária principal será o custo médio por tarefa, incluindo falhas, $\bar{C}_{b,c} = \sum_{i,r} C_{i,c,r}^{\text{API}} / (n_b K)$. A medida complementar será o custo por resolução, $\sum_{i,r} C_{i,c,r}^{\text{API}} / \sum_{i,r} s_{i,c,r}$, indefinido quando não houver resolução. O numerador incluirá todas as execuções. Infraestrutura e ferramentas serão informadas separadamente; uma redução apenas no custo de API não será apresentada como redução comprovada do custo operacional total. Tokens, cache, número de chamadas e participação do modelo forte serão diagnósticos de consumo.

A latência será $T_{i,c,r} = t_{i,c,r}^{\text{fim}} - t_{i,c,r}^{\text{inicio}}$, do fornecimento da tarefa ao agente até a entrega ou o encerramento, incluindo esperas, ferramentas, testes internos e coordenação. O tempo de avaliação externa será registrado à parte. A medida principal será a média $\bar{T}_{b,c}$ sobre todas as execuções, incluindo falhas e esgotamentos de tempo; também serão apresentados mediana, percentil 95, distribuição e frequência de encerramentos por limite. A duração de trabalhos paralelos será medida pelo tempo decorrido, e seu custo incluirá todas as chamadas.

A confiabilidade da conclusão interna será descrita por

$$\hat{F}_{b,c} = \frac{\sum_{i,r} d_{i,c,r} (1 - s_{i,c,r})}{\sum_{i,r} d_{i,c,r}}, \quad (3)$$

em que $d_{i,c,r} = 1$ indica declaração de tarefa concluída. O indicador ficará indefinido na ausência dessas declarações. Abandonos e encerramentos sem solução serão apresentados separadamente.

Esse indicador mede a fração de declarações de conclusão rejeitadas pelo avaliador oficial. Seu denominador difere do de uma taxa de falsos positivos condicionada a candidatos incorretos; ele não estima o erro de cada julgamento intermediário nem a correção semântica além do benchmark. Sua comparação entre condições será acompanhada da frequência de declarações de conclusão e da taxa de resolução, para que declarar menos conclusões não seja interpretado isoladamente como melhoria do sistema.

As expressões acima descrevem as tarefas avaliadas com pesos iguais. Uma estimativa para uma população com probabilidades desiguais de inclusão empregará os pesos do plano amostral, também nas estimativas de custo e tempo. Repetições da mesma tarefa não serão tratadas como novas tarefas independentes.

Os diagnósticos incluirão custo, tempo e chamadas por atividade, modelo empregado, extensão do contexto, etapas ativadas e descartadas e número de candidatos gerados e julgados. Esses registros permitirão distinguir economia local de redução do gasto total: contexto menor pode gerar mais consultas, e uma etapa adicional pode evitar reparos posteriores. Métricas de localização baseadas nos arquivos da solução de referência serão calculadas somente após a execução e interpretadas como aproximações, pois soluções alternativas podem modificar outros arquivos. Elas não substituirão a resolução final nem definirão retrospectivamente o contexto ideal disponível ao agente.

G. Critérios de benefício e aceitação

Qualidade, custo e tempo serão analisados conjuntamente, sem redução a um escore único. Para uma configuração c e a referência forte c_0 , no mesmo benchmark, definem-se a diferença de resolução $\Delta R_c = R_{b,c} - R_{b,c_0}$, a economia relativa $E_c = 1 - \bar{C}_{b,c} / \bar{C}_{b,c_0}$ e a razão de latência $L_c = \bar{T}_{b,c} / \bar{T}_{b,c_0}$, para denominadores positivos. As estimativas desses contrastes respeitarão o pareamento por tarefa.

O protocolo adota dois cenários suficientes de benefício econômico: redução de custo com taxa de resolução preservada ou superior ($E_c > 0$ e $\Delta R_c \geq 0$); e economia de pelo menos 90% com perda de até cinco pontos percentuais ($E_c \geq 0,90$ e $\Delta R_c \geq -0,05$). Em ambos, o limite operacional inicial será $L_c \leq 2$. O segundo cenário corresponde, por exemplo, a reduzir o custo médio de US\$ 5 para US\$ 0,50 por tarefa, admitindo uma queda de resolução de 60% para 55%. Esses valores expressam critérios de projeto, não resultados experimentais nem limites estabelecidos pela literatura.

Os cenários não impõem economia mínima de 90% a toda configuração nem autorizam perda de cinco pontos para qualquer economia. Outras combinações serão apresentadas como trocas entre dimensões, sem aceitação automática ou interpolação de uma tolerância não especificada. Configurações dominadas, isto é, sem vantagem em nenhuma das três dimensões e piores em pelo menos uma, serão identificadas; configurações não dominadas não serão, por esse fato, consideradas aceitáveis. O limite de latência se aplica à média e será acompanhado pelo percentil 95 e pelo teto individual de execução, evitando interpretar aprovação média como garantia para cada tarefa.

Os requisitos de cada cenário deverão ser satisfeitos pela mesma configuração na mesma comparação. A análise empregará intervalos de confiança com cobertura conjunta de 95% para os contrastes confirmatórios, com procedimento de construção declarado antes da avaliação final. A aceitação exigirá que os limites inferiores sustentem os requisitos de resolução e economia e que o limite superior da razão de latência não exceda dois. Uma estimativa pontual favorável será considerada inconclusiva quanto à aceitação se sua incerteza não permitir sustentar esses requisitos. A ausência de diferença significativa não demonstrará preservação de qualidade. O cenário de perda limitada permitirá analisar não inferioridade com margem de cinco pontos, mas a aceitação exigirá também satisfazer o requisito de economia correspondente e a restrição de tempo.

Os contrastes com os demais controles serão apresentados nas mesmas dimensões. Um benefício frente ao harness padrão estabelecerá vantagem apenas frente àquela referência; vantagens adicionais atribuídas à complexidade da abordagem dependerão das comparações com economia de tokens, substituição do modelo e escalonamento simples.

H. Piloto, dimensionamento e análise final

O piloto será realizado em tarefas de desenvolvimento e terá cinco entregas: estimativas de custo e duração por condição; variabilidade entre repetições e frequência de resultados

discordantes entre condições na mesma tarefa; validação da entrega e dos registros; diagnóstico de falhas externas e sinais observáveis de dificuldade; e cenários de orçamento para a avaliação final. Esses dados orientarão o tamanho da amostra, o número de repetições, os limites de gasto, tempo e passos e o tratamento de falhas externas. A regra do escalonamento simples será ajustada apenas nessa etapa. Tentativas internas, repetições experimentais e passos terão limites distintos.

A triagem de variantes poderá usar tarefas selecionadas por dificuldade histórica, conforme a distinção estabelecida na introdução [10]. Seus resultados orientarão o desenvolvimento, sem serem tratados como estimativas do benchmark inteiro. A avaliação confirmatória empregará tarefas reservadas e um plano de seleção previamente registrado. Versões, recortes e alocação entre benchmarks permanecerão decisões de dimensionamento; a seleção atual dos três conjuntos será mantida até essa definição.

O dimensionamento considerará a precisão dos contrastes pareados de resolução, custo e tempo, a dependência entre tarefas de um mesmo repositório e a variabilidade entre repetições. Antes da avaliação final serão congelados as configurações, os controles, as ablações pertinentes, os identificadores das tarefas, o plano amostral, os tetos de recursos, o método de construção dos intervalos e a regra de decisão conjunta. O piloto informará a viabilidade estatística e financeira; os critérios de aceitação não serão relaxados apenas para acomodar resultados favoráveis a uma configuração. Se houver precisão insuficiente, serão reduzidas variantes exploratórias, ampliada a avaliação ou limitada a conclusão, sem declarar benefício demonstrado com base apenas na estimativa pontual.

Os resultados serão apresentados separadamente por benchmark. No Multi-SWE-bench serão também desagregados por linguagem e repositório, acompanhados do tamanho dos grupos; esses contrastes serão descritivos quando a amostra não sustentar inferência própria. Patches, logs e diagnósticos permitirão examinar onde custo, tempo e resolução se alteraram. Uma auditoria complementar de requisitos não cobertos pelos testes oficiais só sustentará alegações adicionais se seu procedimento, amostra e critérios forem definidos previamente, com revisão sem identificação da configuração. A avaliação principal permanecerá vinculada ao verificador oficial.

III. HIPÓTESES DE MECANISMOS PARA A ABORDAGEM

A questão de projeto é quais intervenções permitem satisfazer os critérios da Seção II. O espaço de investigação inclui composição e roteamento de modelos, engenharia de contexto, adaptação do processo e verificação. As revisões de Wang et al. e Ferrag et al. situam planejamento, contexto, ferramentas e avaliação como componentes dos sistemas de agentes, sem estabelecer uma configuração universalmente superior [20], [21]. Aqui, esses componentes serão examinados por seus efeitos sobre resolução e recursos, mantendo os pesos dos modelos fixos e sem pressupor a necessidade de uma arquitetura multiagente.

A. Atividades, modelos e controle do processo

Uma atividade, como localizar código ou validar um patch, não exige necessariamente um agente autônomo ou um modelo exclusivo. A composição fixa atribui modelos a atividades previamente definidas; o roteamento de modelos escolhe qual modelo acionar a partir dos sinais disponíveis. O roteamento de processo decide quais etapas executar, enquanto a seleção de contexto determina a informação fornecida a cada etapa. Essas decisões podem interagir, mas suas contribuições não serão confundidas com o número de agentes.

FlowGen organiza agentes segundo atividades de desenvolvimento e compara processos inspirados em Waterfall, TDD e Scrum. Em HumanEval e MBPP, a remoção de testing prejudicou fortemente a correção funcional, enquanto os efeitos de outras atividades foram menores ou inconsistentes para essa medida; design e revisão também afetaram indicadores de qualidade estática [22]. Esses resultados sustentam avaliar atividades separadamente, mas sua transferência de problemas predominantemente algorítmicos para issues em repositórios permanece limitada. Já Agentless utiliza um processo fixo de localização, reparo e validação, sem delegar ao modelo a escolha livre da próxima etapa. Sua configuração gera múltiplos patches e testes candidatos, mostrando que simplicidade de controle não implica uma única chamada ou baixo consumo por definição [11].

A adequação do processo depende também do modelo. No SWE-Gym, os modelos-base Qwen de 7B e 32B, sem o ajuste fino estudado, resolvem respectivamente 1,0% e 3,0% do SWE-bench Lite com OpenHands, contra 7,0% e 19,0% com MoatlessTools, cujo workflow restringe o espaço de ações [23]. O contraste envolve scaffolds completos e não isola a restrição de autonomia. Ele motiva investigar se um processo mais adequado permite aproveitar melhor modelos de menor custo. O mesmo trabalho estuda treinamento de agentes e verificadores; disponibilizar instruções procedurais ou ferramentas constitui uma intervenção em inferência e não será apresentado como substituto experimentalmente equivalente ao ajuste fino.

B. Composição de modelos por atividade

Jiang et al. distinguem DirectSolve, que gera uma solução a partir de um contexto amplo, de SelectSolve, que separa localização de arquivos e reparo. No SWE-bench Verified, DirectSolve com Gemini 1.5 Pro alcança 38,0% de resolução, e SelectSolve com esse mesmo modelo nas duas etapas alcança 39,2%. Mantendo Gemini na seleção e empregando Claude 3.7 Sonnet no reparo, SelectSolve alcança 48,6% [12]. O contraste motiva a especialização por atividade, mas não demonstra vantagem econômica nem a eficácia de um roteador dinâmico. Os resultados principais incluem amostragem de oito patches e seleção de uma entrega; portanto, não representam a acurácia de uma única geração sem processamento adicional.

A vantagem da separação não é uniforme. Em uma ablação com 100 tarefas, DirectSolve e SelectSolve obtêm, respectivamente, 51% e 49% com Gemini 2.5 Pro, enquanto a separação apresenta estimativas maiores para os demais modelos examinados. O estudo também reporta custos agregados

superiores para seus métodos de contexto longo, sem fornecer uma comparação detalhada de custo e latência entre todas as composições [12]. Assim, a escolha do modelo e a organização do contexto serão investigadas conjuntamente, sem pressupor que decompor atividades ou combinar provedores seja necessário.

A hipótese inicial de distribuição de papéis combina execução predominantemente por um modelo de menor custo com planejamento, revisão ou diagnóstico por um modelo mais capaz. Uma alternativa inverte essa distribuição: um modelo de menor custo prepara o contexto e o mais capaz implementa a solução. Uma variante empregaria um orquestrador para acompanhar dependências, resultados intermediários, orçamento e encerramento. Essas hipóteses permanecem abertas: especialização e assistência podem melhorar a execução, mas também introduzir custo e falhas de transferência de contexto. Gatilhos de escalonamento, conteúdo da assistência e limites de intervenção são decisões do mecanismo a investigar.

Antes de justificar essa assistência, é necessário compará-la com uma alternativa mais simples: permitir que o modelo barato tente resolver a tarefa novamente. Essa comparação envolve três questões: se o modelo consegue gerar uma solução correta, se o sistema consegue reconhecê-la e se os recursos necessários compensam o benefício obtido.

C. Gerar uma solução correta

Repetir a geração produz várias respostas candidatas para a mesma tarefa, por exemplo, diferentes propostas de alteração do código. Algumas podem estar corretas e outras não. Em um caso idealizado, com tentativas independentes e probabilidade fixa $p > 0$ de acerto em cada uma, a probabilidade de haver ao menos uma solução correta entre as k respostas geradas é $1 - (1 - p)^k$. Essa probabilidade converge a um quando o número de tentativas cresce indefinidamente. A conclusão diz respeito à existência de uma solução correta entre as respostas candidatas: a expressão não informa qual delas está correta nem como escolhê-la para entrega. Se a probabilidade de acerto for zero, repetir a mesma estratégia de geração não resolve a tarefa. Independência e probabilidade fixa são hipóteses desse caso idealizado, não propriedades asseguradas de um agente que modifica o contexto e recebe orientações entre tentativas.

D. Reconhecer a solução correta

Para transformar as respostas candidatas em uma entrega, o sistema precisa de um critério de seleção. Testes podem fornecer esse critério, mas uma solução incorreta pode ser aprovada quando o erro não é coberto pelos testes. Essa aprovação indevida constitui um falso positivo. Stroebl et al. observaram, em HumanEval e MBPP com testes de cobertura limitada, que modelos com menor acurácia em uma única resposta também produziram mais falsos positivos. Sob os verificadores estudados, repetir a geração até obter aprovação apresentou um limite de acurácia mesmo com orçamento infinito [24]. Assim, encontrar uma resposta que passa pelo

verificador não equivale necessariamente a encontrar uma resposta correta.

Um caso idealizado explicita essa diferença. Mantendo a probabilidade de acerto p , suponha que os pares de geração e julgamento sejam independentes entre tentativas, com probabilidades fixas. Sejam t a probabilidade de o verificador aceitar uma solução correta e f a probabilidade de aceitar uma incorreta. Ao entregar a primeira resposta aprovada, a probabilidade de essa entrega estar correta, no limite de tentativas, é

$$\frac{pt}{pt + (1 - p)f}, \quad (4)$$

quando o denominador é positivo. O numerador representa a probabilidade de gerar uma solução correta e aceitá-la; o denominador inclui também as aceitações de soluções incorretas. Para $0 < p < 1$ e $f > 0$, o limite é inferior a um. Se o verificador não aceitar soluções incorretas ($f = 0$) e houver chance positiva de gerar e aceitar uma correta ($pt > 0$), o limite será um. Portanto, a confiabilidade da seleção depende tanto do gerador quanto do verificador.

Se é necessário reconhecer erros para decidir quando tentar novamente, uma possibilidade é pedir ao próprio modelo que revise sua resposta. Entretanto, Huang et al. encontraram dificuldades na autocorreção de raciocínio quando a revisão depende apenas do modelo, sem feedback externo; em alguns casos, as revisões pioraram o desempenho [25]. Esses resultados não estabelecem impossibilidade de autocorreção em programação: executar o código e observar falhas em testes fornece evidências externas que não estão presentes naquele cenário. A capacidade de produzir uma resposta não deve, portanto, ser tomada como garantia de conseguir avaliar e corrigir essa mesma resposta.

Outra possibilidade é usar um modelo mais capaz como juiz, isto é, encarregá-lo de avaliar a resposta produzida pelo modelo barato. Um juiz que aceite menos soluções incorretas pode melhorar a seleção, mas maior capacidade de geração não assegura julgamento infalível. Também há possíveis dependências entre produção e avaliação: em sumarização, Panickssery et al. identificaram reconhecimento e preferência pelas próprias gerações [26]. A transferência desse resultado para código permanece uma questão empírica, e usar modelos diferentes não assegura que seus erros sejam independentes.

O acesso a evidências pode ser tão relevante quanto a escolha do revisor. Agent-as-a-Judge emprega ferramentas e inspeção dos artefatos para verificar requisitos de desenvolvimento de software [27]. Em busca, sistemas de dados e interfaces gráficas, AJ-Bench encontra ganhos com avaliação baseada em ambiente, mas também falhas na aquisição de evidências e na interpretação dos resultados [28]. Esses trabalhos motivam revisão apoiada em testes e artefatos disponíveis, sem presumir que cada etapa adicional compense seu custo ou que seus resultados se transfiram diretamente para resolução de issues.

E. Decidir se a assistência compensa

Mesmo quando repetição e revisão aumentam a chance de uma entrega correta, cada tentativa, teste e chamada ao

juiz consome recursos. Orçamento ilimitado é uma condição teórica; a questão operacional é quanto benefício pode ser obtido dentro dos limites de custo e latência. No estudo de Stroebel et al., o número de tentativas de maior utilidade dependeu da penalização atribuída aos falsos positivos [24]. Não decorre disso um número universal de repetições, nem que poucas tentativas sejam economicamente desvantajosas. É necessário comparar seu consumo e sua resolução com a execução direta pelo modelo forte e com outras formas de assistência.

Essa assistência pode ir além do julgamento. Um modelo forte pode diagnosticar a falha, orientar uma nova tentativa ou modificar o código. Nesses casos, ele intervém na produção das soluções, alterando as condições do modelo de repetição com probabilidade fixa. O mecanismo deverá distinguir nova geração, revisão de uma resposta existente, reparo e escalonamento, entendido como transferência de parte do trabalho ao modelo forte. A ajuda pode melhorar a solução, mas também consumir mais recursos ou introduzir erros ao transferir informações entre modelos.

Após uma revisão, continuar reparando tampouco garante melhoria. Uma alteração pode corrigir uma falha ou tornar incorreta uma solução antes correta. VRR-Stop distingue essas possibilidades dos erros do verificador e propõe decidir pela continuação com base na melhoria esperada da próxima rodada [29]. Seus resultados em raciocínio, incluindo cenários de estresse com perturbações controladas, fundamentam investigar regras de parada, sem estabelecer ganhos equivalentes em programação. Aqui, a hipótese é que os sinais disponíveis durante a execução permitam escolher entre repetir, reparar, escalar ou encerrar, obtendo benefício suficiente para compensar o gasto adicional e o risco de piorar a solução.

Portanto, ampliar o número de tentativas do modelo barato não assegura, por si só, que o sistema entregue uma solução correta ou alcance a taxa de resolução do modelo forte. O benefício depende também da capacidade de reconhecer erros e acertos e dos recursos necessários para isso. A assistência de outro modelo constitui uma hipótese para melhorar essa relação, a ser comparada com repetição simples e execução direta pelo modelo forte. Sua adoção só se justificará se a combinação de resolução, custo e latência atender aos critérios do protocolo; uma melhoria de resolução isolada não demonstrará vantagem econômica.

F. Aquisição, seleção e distribuição de contexto

Engenharia de contexto compreende, neste estudo, a aquisição, seleção, representação, distribuição e atualização da informação disponível durante a execução. Seu efeito pode decorrer tanto de incluir evidências ausentes quanto de reduzir interferência ou consultas redundantes. Em uma análise controlada de DirectSolve, mantendo os arquivos-alvo no início do prompt, ampliar o contexto de 100 mil para um milhão de tokens reduziu a medida reportada de pass@1 de 39% para 31% [12]. Como a informação necessária já estava presente, o resultado evidencia uma limitação no aproveitamento de con-

texto adicional, mas não demonstra que um seletor operacional consiga identificar previamente esses arquivos.

A seleção também pode preservar hipóteses alternativas. Em Agentless, gerar 40 patches a partir de localizações reunidas em um único contexto resolveu 85 issues; distribuir o mesmo número de candidatos entre quatro conjuntos de localização resolveu 96. Os custos diferiram e as configurações não isolam uma causa única, mas o contraste motiva investigar contextos alternativos em vez de agregar toda informação recuperada [11]. Cobertura e concisão serão tratadas conjuntamente: excluir informação necessária pode reduzir tokens e, ao mesmo tempo, comprometer a solução ou provocar novas consultas.

Conhecimento procedural adicional apresenta limitações semelhantes. Nos resultados preliminares do SWE-Skills-Bench, com Claude Code e Haiku 4.5, 39 de 49 skills não alteraram a taxa de aprovação; 24 dessas já alcançavam 100% sem skill. Algumas reduziram tokens sem alterar a aprovação, enquanto outras aumentaram o consumo ou prejudicaram o resultado. A média passou de 89,8% para 91,0%, com efeitos heterogêneos entre skills [30]. O estudo disponibiliza uma skill previamente selecionada por tarefa e deixa seleção dinâmica e composição de várias skills como questões futuras. A hipótese aqui é que selecionar orientação compatível com tarefa, versão e contexto possa melhorar a relação entre resolução e recursos; sua eficácia não será inferida apenas da pertinência temática da skill.

A consulta a estruturas do código oferece outro mecanismo de aquisição. O icat-agent emprega ferramentas de inspeção de símbolos e cadeias de chamadas, mas seus resultados não isolam o benefício dessas ferramentas [13]. CodeGraph será considerado como candidato de implementação para uma variante de recuperação estrutural, sujeito à verificação de suporte e à comparação com a busca da configuração de referência. A avaliação deverá distinguir custo de indexação, atualização e consulta. A integração de ferramentas via MCP, se adotada, será uma decisão de interface; seu uso não constituirá evidência de melhoria na seleção ou no raciocínio.

G. Ativação de etapas e verificação intermediária

O icat-agent adapta o processo conforme uma avaliação da descrição da issue: ativa um Explorer quando faltam informações e, nos demais casos, inicia diretamente edição e validação em paralelo. Dispensar o Explorer não elimina a inspeção do repositório pelo editor. Na ablação com 60 issues bem especificadas por benchmark, forçar exploração elevou o custo e não aumentou a resolução agregada. Os subconjuntos continham 30 casos originalmente resolvidos e 30 não resolvidos, de modo que os efeitos não estimam diretamente a vantagem da política no benchmark completo [13]. Essa evidência motiva uma hipótese restrita: ativar uma etapa dedicada de exploração somente quando seus benefícios esperados compensarem o trabalho adicional, comparando essa política com processos fixos.

O mesmo sistema separa contextos de edição e validação e restringe a comunicação a resultados estruturados, evitando compartilhar diretamente o código dos testes com o editor

e o histórico de raciocínio do editor com o validador [13]. Essa separação pode reduzir dependências indesejadas, mas não garante independência estatística dos erros. A hipótese de que o isolamento reduza aceitações indevidas será examinada por contrastes que explicitem a informação compartilhada, preservando o avaliador reservado como medida externa.

Verificar premissas antes de implementar constitui uma intervenção distinta. Spec Kit Agents introduz consultas ao repositório e verificações de artefatos intermediários. Na comparação entre os workflows Full e Full-Augmented, a nota média do juiz LLM passou de 3,51 para 3,66, enquanto o tempo reportado para pares concluídos passou de 24,0 para 37,2 minutos. A pequena avaliação humana, com seis tarefas, registrou mais preferências pelo Full do que pelo Full-Augmented, além de uma maioria de empates [31]. Portanto, grounding intermediário motiva investigação, sem demonstrar benefício uniforme entre avaliadores ou justificar sua execução em toda tarefa.

A taxonomia de Macedo distingue especificação, contexto, papéis, execução, validação e portabilidade em frameworks de desenvolvimento; sua comparação documental, pontuada por um único avaliador, não mede superioridade experimental de qualidade ou produtividade [32]. Para esta pesquisa, artefatos intermediários poderão organizar decisões, mas suas afirmações precisarão ser confrontadas com requisitos, dependências e evidências de execução. A existência de um plano coerente não será tratada como confirmação de sua compatibilidade com o repositório.

H. Hipóteses e seleção da abordagem

As evidências motivam investigar três relações: se a composição de modelos por atividade supera alternativas de modelo único; se a adequação e distribuição do contexto permitem reduzir recursos preservando resolução; e se a ativação seletiva de etapas supera processos fixos. São hipóteses sobre a abordagem a desenvolver, e não resultados estabelecidos pelos estudos anteriores. Também permanece aberta a hipótese de que melhorar o contexto antes de escalar o modelo resolva parte das falhas com menor custo. Sua avaliação dependerá de sinais disponíveis durante a execução e incluirá o gasto de obter esses sinais.

A paralelização permanece uma hipótese para reduzir latência em subtarefas sem dependências conflitantes. Integração, verificação e coordenação consumirão recursos e poderão anular essa vantagem. Revisores baseados em modelo também serão investigados como mecanismos internos, mantendo a avaliação externa como medida de resolução. O benefício de revisar dependerá tanto das falhas detectadas quanto do gasto adicional e de possíveis regressões.

Permanece em aberto se a contribuição relativa do scaffold, a estrutura de ferramentas e controle que organiza a atuação do modelo, diminui com o aumento da capacidade do modelo. O estudo em GAIA não encontrou redução monotônica no nível mais difícil e observou diferenças entre famílias; seu domínio não permite antecipar a direção dessa interação nas tarefas de código aqui consideradas [4]. Essa hipótese será

examinada apenas na medida em que os modelos e as ablações implementadas permitirem contrastes adequados.

A seleção dos mecanismos deverá justificar cada componente por uma hipótese de efeito sobre resolução, custo ou tempo e permitir sua remoção em ablações pertinentes. A arquitetura resultante será uma intervenção submetida ao protocolo, e sua eficiência permanecerá uma hipótese até a avaliação reservada.

IV. DISCUSSÃO E AMEAÇAS À VALIDADE

Os controles propostos reduzem algumas fontes de confusão, mas não eliminam as limitações de exposição prévia e representatividade. Registrar datas e reservar tarefas para avaliação controla o ajuste da abordagem às instâncias observadas; não demonstra ausência dessas tarefas no treinamento dos modelos. Diferenças entre conjuntos e linguagens também podem refletir composição de repositórios, dificuldade e domínio. A interpretação permanecerá vinculada à população efetivamente representada pelo plano amostral.

Os testes oficiais medem uma forma operacional de correção. A distinção entre executar um programa e decidir se seu comportamento é correto constitui o problema do oráculo de testes [33]. EvalPlus identificou soluções incorretas aceitas pelos testes originais de HumanEval, enquanto SWE-ABS encontrou limitações semelhantes na discriminação de patches em benchmarks de repositórios [34], [35]. Esses achados justificam delimitar o significado da resolução medida; suas taxas não serão extrapoladas para os conjuntos deste estudo. Patches aprovados podem violar requisitos não cobertos, enquanto soluções alternativas podem ser rejeitadas por testes dependentes da implementação de referência, problemas discutidos por Gorinova et al. [5]. Registros de falhas e auditorias complementares ajudam a investigar esses casos, mas não transformam a pontuação em garantia geral de segurança ou manutenibilidade. Alegações sobre essas propriedades exigirão medidas próprias.

A referência prática será o sistema completo formado por modelo e harness. Diferenças de ferramentas internas, prompts ou gestão de contexto podem integrar a intervenção e impedir a atribuição causal a um componente isolado. As ablações permitirão restringir essa atribuição quando os demais fatores puderem ser mantidos. A comparação com o modelo forte acrescido de economia de tokens é particularmente relevante: um ganho frente ao uso padrão pode já ser alcançado por uma intervenção mais simples. Diferenças residuais de suporte entre modelos e harnesses serão limitações explícitas dos contrastes.

Modelos, serviços e tarifas podem variar durante a coleta. Versões registradas e ordem de execução aleatorizada reduzem a confusão temporal, sem garantir estabilidade do serviço. Custos de API estimados por tokens não equivalem automaticamente a despesas de assinatura ou custo operacional total. A separação desses valores delimita o significado da economia observada, inclusive quando uma redução de chamadas exige mais infraestrutura local.

O estudo controlado em GAIA encontrou efeitos de scaffold dependentes de modelo e dificuldade, inclusive sem a redução monotônica esperada para modelos mais capazes no nível difícil [4]. Nesse nível, a vantagem do scaffold com múltiplos papéis apareceu com maior clareza na família Anthropic, mas não nos modelos de outros provedores examinados. Os scaffolds estruturados também fizeram menos chamadas de ferramenta, terminaram mais rapidamente em média e recuperaram-se com maior frequência de erros intermediários; isso mostra que acrescentar etapas de organização não implica necessariamente mais ações ou maior gasto. Ainda assim, sua transferência para engenharia de software é limitada pelo domínio avaliado e pelo fato de o controle ReAct usar uma superfície de ferramentas diferente da dos outros scaffolds. Já Vats e Golev observaram grande variação em tokens por tarefa resolvida entre harnesses de programação, mas a amostra de 50 tarefas e diferenças nos controles de turnos limitam a generalização do fator relatado para custo monetário ou para os modelos futuros [2]. Portanto, resultados anteriores justificam medir a interação modelo–scaffold e o consumo efetivo, sem fixar sua magnitude no presente estudo.

A evidência sobre processos também depende da tarefa e da medida adotada. FlowGen estuda principalmente problemas algorítmicos; Agentless e icat-agent avaliam issues; Spec Kit Agents inclui features e julgamento por LLM [11], [13], [22], [31]. Seus ganhos não compõem uma comparação direta entre arquiteturas. Da mesma forma, a ausência de ganho de uma skill pode refletir saturação da aprovação, e um ganho de localização pode não se converter em reparo correto [12], [30]. A interpretação deverá distinguir cobertura de contexto, qualidade de artefatos intermediários e correção operacional da entrega.

Uma descrição detalhada da issue não garante que suas sugestões estejam corretas, e sinais de falha podem refletir falta de contexto, limitação do modelo ou problemas de ambiente. Políticas condicionais podem, portanto, dispensar uma etapa necessária ou ativar trabalho redundante. A comparação com processos fixos medirá o efeito líquido dessas decisões, incluindo seu custo. Isolar contextos de editor e verificador tampouco assegura independência dos erros, pois ambos podem compartilhar premissas incorretas sobre os requisitos. Nem a aprovação interna nem o comportamento fail-to-pass de um teste gerado substituirão a avaliação reservada.

A seleção de variantes com dados de desenvolvimento pode produzir configurações ajustadas às tarefas observadas. A avaliação reservada e a distinção entre triagem e confirmação limitam esse problema. Subconjuntos escolhidos por dificuldade não sustentarão automaticamente inferência para todo o benchmark; repetir execuções sobre poucas tarefas tampouco substitui diversidade de tarefas e repositórios. Quando orçamento ou variabilidade impedirem precisão suficiente, a aceitação permanecerá inconclusiva. Uma configuração favorável entre as alternativas avaliadas não demonstrará que foi encontrado o menor processo possível nem que a mesma distribuição de atividades permanece adequada após a troca de modelo.

Os limiares de economia, resolução e latência são critérios de projeto. Uma configuração aceita segundo esses critérios pode ser inadequada em um contexto que exija preservar integralmente a resolução ou imponha prazos mais estritos. O limite de tempo médio também não garante ausência de casos muito lentos. Por isso, serão apresentados os valores separados, sua incerteza e a distribuição de duração, permitindo examinar outras preferências sem redefinir retrospectivamente o sucesso do estudo.

Por fim, resultados em um benchmark, par de modelos ou harness não serão generalizados automaticamente a outros. O protocolo mede recursos consumidos pelo agente, sem estimar o esforço de supervisão e correção humana. Eventual benefício econômico segundo as métricas adotadas não demonstrará, por si só, maior produtividade de desenvolvedores ou superioridade de uma arquitetura em todos os contextos.

V. CONCLUSÃO

As evidências examinadas sustentam que a taxa de resolução, isoladamente, não basta para comparar agentes de programação sob restrições de engenharia. O consumo de recursos, o harness e o procedimento de entrega afetam a interpretação do desempenho; exposição prévia às tarefas, verificadores incompletos e seleção de subconjuntos limitam o alcance das conclusões. Esses fatores fundamentam a avaliação conjunta de qualidade operacional, custo e latência.

Os estudos de arquitetura fundamentam investigar conjuntamente modelos, contexto e processo. Composição fixa por atividade, roteamento condicional e seleção de informação constituem decisões distintas; a vantagem de separá-las ou combiná-las depende da tarefa, da capacidade do modelo e da confiabilidade da verificação. A evidência disponível motiva essas intervenções, sem demonstrar que aumentar o número de agentes, fornecer mais contexto ou adicionar etapas produza benefício uniforme.

O protocolo estabelece como avaliar uma abordagem frente ao modelo no harness padrão e a controles de menor custo, economia de tokens no modelo forte e escalonamento simples. Contrastes de composição, contexto e processos fixos serão associados aos mecanismos efetivamente adotados. O benefício será examinado tanto pela preservação da resolução com economia quanto por uma redução substancial de custo que justifique uma perda delimitada de resolução, sob restrição de tempo. Piloto e desenvolvimento antecederão o congelamento das configurações e a avaliação reservada. Composição e roteamento de modelos, engenharia de contexto, ativação de etapas, revisão e paralelização permanecem hipóteses, sem superioridade experimental demonstrada para a abordagem.

REFERÊNCIAS

- [1] S. Kapoor, B. Stroebel, Z. S. Siegel, N. Nadgir, and A. Narayanan, “AI Agents That Matter,” arXiv:2407.01502, 2024. [Online]. Available: <https://arxiv.org/abs/2407.01502>
- [2] N. Vats and O. Golev, “The scaffold effect in coding agents: Harness choice as a hidden variable in coding-agent evaluation,” arXiv:2607.22585, 2026. [Online]. Available: <https://arxiv.org/abs/2607.22585>

- [3] M. Zheng, K. Han, B. Li, H. Xu, Y. Tian, W. He, H. Zhou, J. Guo, H. Hu, L. Ma, C. Xu, G. Dai, L. Xia, Y. Wei, Y. Wang, and Y. Wang, "Claw-SWE-Bench: A benchmark for evaluating OpenClaw-style agent harnesses on coding tasks," arXiv:2606.12344, 2026. [Online]. Available: <https://arxiv.org/abs/2606.12344>
- [4] J. Starace, "Scaffold effects on GAIA: A controlled comparison," arXiv:2606.08529, 2026. [Online]. Available: <https://arxiv.org/abs/2606.08529>
- [5] M. I. Gorinova, M. Baker, A. Heineke, M. Shaposhnikov, R. Willoughby, and D. Knox, "Position: Coding benchmarks are misaligned with agentic software engineering," arXiv:2606.17799, 2026. [Online]. Available: <https://arxiv.org/abs/2606.17799>
- [6] T. Prathifkumar, N. S. Mathews, and M. Nagappan, "Does SWE-Bench-Verified test agent ability or model memory?" arXiv:2512.10218, 2025. [Online]. Available: <https://arxiv.org/abs/2512.10218>
- [7] L. Zhang, S. He, C. Zhang, Y. Kang, B. Li, C. Xie, J. Wang, M. Wang, Y. Huang, S. Fu, E. Nallipogu, Q. Lin, Y. Dang, S. Rajmohan, and D. Zhang, "SWE-bench Goes Live!" arXiv:2505.23419, 2025.
- [8] ByteDance Seed, "Multi-SWE-bench: A Multilingual Benchmark for Issue Resolving," arXiv:2504.02605, 2025.
- [9] M. A. Merrill *et al.*, "Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces," arXiv:2601.11868, 2026. [Online]. Available: <https://arxiv.org/abs/2601.11868>
- [10] F. Ndzomga, "Efficient benchmarking of AI agents," arXiv:2603.23749, 2026. [Online]. Available: <https://arxiv.org/abs/2603.23749>
- [11] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," arXiv:2407.01489, 2024. [Online]. Available: <https://arxiv.org/abs/2407.01489>
- [12] M. Jiang, Y. Ruan, L. Lastras, P. Kapanipathi, and T. Hashimoto, "Putting it all into context: Simplifying agents with LCLMs," arXiv:2505.08120, 2025. [Online]. Available: <https://arxiv.org/abs/2505.08120>
- [13] Y. Chen, A. Ahmad, Y. Zhou, and R. Jabbarvand, "Unlocking model potentials through adaptive multi-agent scaffolding for efficient issue resolution," arXiv:2606.25514, 2026. [Online]. Available: <https://arxiv.org/abs/2606.25514>
- [14] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can Language Models Resolve Real-World GitHub issues?" in *International Conference on Learning Representations*, 2024.
- [15] M. S. Rashid *et al.*, "SWE-PolyBench: A multi-language benchmark for repository level evaluation of coding agents," arXiv:2504.08703, 2025.
- [16] R. Marten, "Terminal-Bench 4.0: Calibrating task resources, fixing tasks, and removing saturated tasks," Terminal-Bench blog, Aug. 2026. [Online]. Available: <https://www.tbench.ai/news/terminal-bench-4-0>
- [17] OpenAI, "Por que o SWE-bench Verified não mede mais capacidades de codificação de ponta?" Feb. 2026. [Online]. Available: <https://openai.com/pt-BR/index/why-we-no-longer-evaluate-swe-bench-verified/>
- [18] —, "Separando sinal de ruído em avaliações de programação," Jul. 2026. [Online]. Available: <https://openai.com/pt-BR/index/separating-signal-from-noise-coding-evaluations/>
- [19] Q. Zhou *et al.*, "FeatureBench: Benchmarking agentic coding for complex feature development," in *International Conference on Learning Representations*, 2026.
- [20] H. Wang, J. Gong, H. Zhang, J. Xu, and Z. Wang, "AI agentic programming: A survey of techniques, challenges, and opportunities," arXiv:2508.11126, 2025. [Online]. Available: <https://arxiv.org/abs/2508.11126>
- [21] M. A. Ferrag, N. Tihanyi, and M. Debbah, "From LLM reasoning to autonomous AI agents: A comprehensive review," *IEEE Access*, vol. 14, 2026.
- [22] F. Lin, D. J. Kim, and T.-H. Chen, "SOEN-101: Code generation by emulating software process models using large language model agents," arXiv:2403.15852, 2024. [Online]. Available: <https://arxiv.org/abs/2403.15852>
- [23] J. Pan, X. Wang, G. Neubig, N. Jaitly, H. Ji, A. Suhr, and Y. Zhang, "Training software engineering agents and verifiers with SWE-Gym," arXiv:2412.21139, 2025. [Online]. Available: <https://arxiv.org/abs/2412.21139>
- [24] B. Stroebel, S. Kapoor, and A. Narayanan, "The limits of inference scaling through resampling," in *International Conference on Learning Representations*, 2026.
- [25] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large language models cannot self-correct reasoning yet," arXiv:2310.01798, 2023. [Online]. Available: <https://arxiv.org/abs/2310.01798>
- [26] A. Panickssery, S. R. Bowman, and S. Feng, "LLM evaluators recognize and favor their own generations," arXiv:2404.13076, 2024. [Online]. Available: <https://arxiv.org/abs/2404.13076>
- [27] M. Zhuge, C. Zhao, D. R. Ashley, W. Wang, D. Khizbullin, Y. Xiong, Z. Liu, E. Chang, R. Krishnamoorthi, Y. Tian, Y. Shi, V. Chandra, and J. Schmidhuber, "Agent-as-a-judge: Evaluate agents with agents," arXiv:2410.10934, 2024. [Online]. Available: <https://arxiv.org/abs/2410.10934>
- [28] W. Shi, Y. Wang, Y. Zhao, Y. Chen, F. Feng, X. Hao, X. Su, Q. Gu, H. Su, X. Cai, and X. He, "AJ-Bench: Benchmarking agent-as-a-judge for environment-aware evaluation," arXiv:2604.18240, 2026. [Online]. Available: <https://arxiv.org/abs/2604.18240>
- [29] Y. Wu, S. Shen, R. Yang, H. Peng, and B. Hu, "Verify, repair, repeat, or stop? robust stopping for noisy verify-repair loops in LLM agents," arXiv:2607.17641, 2026. [Online]. Available: <https://arxiv.org/abs/2607.17641>
- [30] T. Han, Y. Zhang, W. Song, C. Fang, Z. Chen, Y. Sun, and L. Hu, "SWE-Skills-Bench: Do agent skills actually help in real-world software engineering?" arXiv:2603.15401, 2026. [Online]. Available: <https://arxiv.org/abs/2603.15401>
- [31] P. Taghavi and S. Bhavani, "Spec Kit Agents: Context-grounded agentic workflows," arXiv:2604.05278, 2026. [Online]. Available: <https://arxiv.org/abs/2604.05278>
- [32] S. O. d. Macedo, "From prompt to process: a process taxonomy and comparative assessment of frameworks supporting AI software development agents," arXiv:2606.04967, 2026. [Online]. Available: <https://arxiv.org/abs/2606.04967>
- [33] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, 2015.
- [34] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation," arXiv:2305.01210, 2023. [Online]. Available: <https://arxiv.org/abs/2305.01210>
- [35] B. Yu, Y. Cao, Y. Zhang, L. Lin, J. Xu, Z. Zhong, Q. Xu, G. Wang, J. Cao, S.-C. Cheung, P. He, and L. Briand, "SWE-ABS: Adversarial benchmark strengthening exposes inflated success rates on test-based benchmark," arXiv:2603.00520, 2026. [Online]. Available: <https://arxiv.org/abs/2603.00520>